

Numerical Derivatives Lab Report

Songhang Li

Department of Physics and Astronomy

1 Motivation

Analytical method does not always applicable in the real world physical scenario. With the limited measurements, some data set are discrete and not being able to build a math model analytically. In order to find the rate of change in the quantity of interests, the numerical derivatives method were developed.

2 Theory

2.1 Discrete Derivatives

Analytical derivative:

$$v(t) = \frac{d}{dt}x(t)$$

However, the function of time in LoggerPro is a discrete set of points t_i and x_i . By the definition of derivative:

$$v = \dot{x} = \frac{dx}{dt} = \lim_{\Delta t \rightarrow 0} \frac{x(t + \Delta t) - x(t)}{\Delta t}$$

$\dot{x}$ denotes the derivative of the variable x with respect to time. Applying this in the data set gives the velocity, but variation of the calculation is considered as follow.

2.2 Forward Difference Derivative(FDD)

$$v_{\text{FDD},i} = \dot{x}_{\text{FDD},i} = \frac{x_{i+1} - x_i}{\Delta t}$$

This method takes the function values at the current and next points. It approximates the derivative by the slope of the line through these points.

2.3 Backward Difference Derivative (BDD)

Similar to FDD but uses the current and previous function values to approximate the derivative:

$$v_{\text{BDD},i} = \dot{x}_{\text{BDD},i} = \frac{x_i - x_{i-1}}{\Delta t}$$

2.4 Center Divided Difference (CDD)

This method provides a more accurate approximation by using the average slope between the previous and next function value.

$$\begin{aligned} \dot{x}_{\text{CDD},i} &= \frac{1}{2} (\dot{x}_{\text{FDD},i} + \dot{x}_{\text{BDD},i}) \\ &= \frac{1}{2} \left(\frac{x_{i+1} - x_i}{\Delta t} + \frac{x_i - x_{i-1}}{\Delta t} \right) = \frac{x_{i+1} - x_{i-1}}{2\Delta t} \end{aligned}$$

2.5 Taylor expansion for CDD and Five Points derivative

1. Start with the Taylor series expansion around a point x_i for f_{i+1} and f_{i-1} :

$$\begin{aligned} f_{i+1} &= f_i + f'_i \Delta x + \frac{f''_i (\Delta x)^2}{2!} + \frac{f^{(3)}_i (\Delta x)^3}{3!} + \dots \\ f_{i-1} &= f_i - f'_i \Delta x + \frac{f''_i (\Delta x)^2}{2!} - \frac{f^{(3)}_i (\Delta x)^3}{3!} + \dots \end{aligned}$$

Subtracting the second equation from the first:

$$f_{i+1} - f_{i-1} = 2f'_i \Delta x + 2\frac{f^{(3)}_i (\Delta x)^3}{3!} + \dots$$

Solve for f'_i , neglect higher-order terms:

$$f'_i \approx \frac{f_{i+1} - f_{i-1}}{2\Delta x}$$

2. Extend the Taylor series and include 2 steps ahead and 2 steps behind:

$$\begin{aligned} f_{i+2} &= f_i + 2f'_i \Delta x + \frac{2^2 f''_i (\Delta x)^2}{2!} + \frac{2^3 f^{(3)}_i (\Delta x)^3}{3!} + \dots \\ f_{i-2} &= f_i - 2f'_i \Delta x + \frac{2^2 f''_i (\Delta x)^2}{2!} - \frac{2^3 f^{(3)}_i (\Delta x)^3}{3!} + \dots \end{aligned}$$

Multiply the CDD equation(in the part1) by 8 and subtract f_{i+2} and f_{i-2} , and rearranging:

$$8(f_{i+1} - f_{i-1}) - (f_{i+2} - f_{i-2}) = 12f'_i \Delta x$$

Solve f'_i that gives the five-point stencil formula:

$$f'_i \approx \frac{-f_{i-2} + 8f_{i-1} - 8f_{i+1} + f_{i+2}}{12\Delta x}$$

3 Method

The packages used are numpy and matplotlib.pyplot. The following code lists are the functions derivFDD(), derivBDD(), derivFDD(), and new function deriv5pt().

```
def derivFDD(x, dt):
    # create an array
    nsteps=len(x)
    xdot=np.empty(nsteps)*np.nan
    # loop through the time steps to
    # define xdot at each step

    for i in range(nsteps-1):
        xdot[i]=(x[i+1] - x[i])/dt
    return xdot
```

Listing 1: derivFDD()

```
def derivBDD(x,dt):
    nsteps=len(x)
    xdot=np.empty(nsteps)*np.nan

    for i in range(1,nsteps):
        xdot[i]=(x[i] - x[i-1])/dt
    return xdot
```

Listing 2: *derivBDD()*

```
def derivCDD(x,dt):
    # create an array of NaNs
    nsteps=len(x)
    xdot=np.empty(nsteps)*np.nan

    for i in range(nsteps):
        if i == 0:
            xdot[i]=(x[i+1]-x[i])/dt #
            the first derivative
            would need the first x
            and second x: x[1]-x[0]
            so we use fdd
        elif i==nsteps-1:
            xdot[i]=(x[i]-x[i-1])/dt#
            bdd
        else:
            xdot[i]=(x[i+1]-x[i-1])/(2*
            dt)
    return xdot
```

Listing 3: *derivCDD()*

```
def deriv5Pts(x, dt):
    nsteps = len(x)
    xdot = np.empty(nsteps) * np.nan

    for i in range(2, nsteps-2):
        xdot[i]=(x[i-2]-8*x[i-1]+8*x[i
        +1]-x[i+2])/(12*dt)
    return xdot
```

Listing 4: *deriv5pt()*

4 Result

4.1 Various numerical approximations

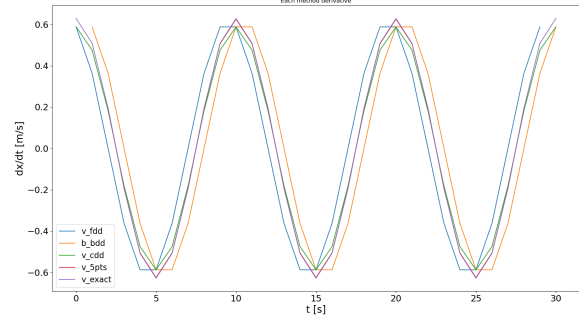
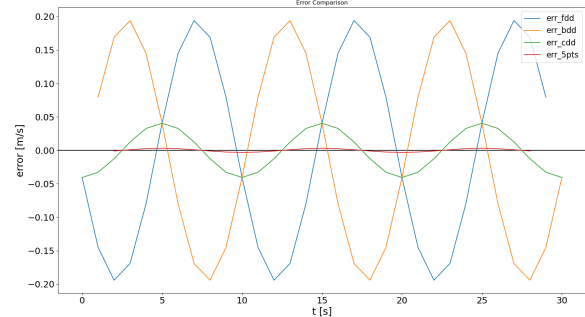
The **Figure 1** shows that the 5pts method is closest to the exact value of the derivative.

4.2 Error vs. time

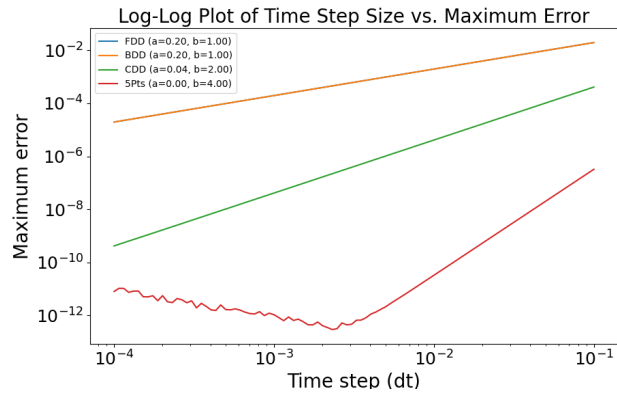
According to the **Figure 2**, it can be seen that unlike other methods that clearly fluctuate up and down from 0, the error of the 5pt method is closest to 0

4.3 Scale of error

The **Figure 3** shows that the data for *FDD* and *BDD* overlap. the *5pts* method has the highest exponent, 4, while the others are 1 or 2. It can also be seen from the orders of the error range that the *5pts* method has the smallest,

Figure 1: Various numerical approximations with $dt = 1$ Figure 2: Error Comparison with $dt = 1$

1×10^{-12} , while the others are 1×10^{-5} or 1×10^{-9} . Thus, the overall performance shows that *FDD* and *BDD* have the relatively high error, the *CDD* after them and the *5pts* the smallest. Therefore, *5pts* is the best approximation among others.

Figure 3: The exponents of *FDD*, *BDD*, *CDD*, *5Pts* are 1, 1, 2, and 4 respectively.

5 Conclusion

While analytical method for a real world is not applicable in most cases, the numerical method with discrete measurement data set can give great enough approximation of the function. According to the analysis in this report, it can be concluded that the *5pt* Derivative method is the best approximation compared to the *FDD*, *BDD*, and *CDD*.